PLACEMENT EDITION · 2027

Network Engineer Placements 2027

From first principles to the offer: networks, routing, systems, code and interviews for Cisco, Qualcomm, Google, Microsoft, Amazon, Juniper, Arista, Nokia and Broadcom.

Working edition · built 29 October 2026 · information correct as of this date

CONTENTS

Contents

0 How to Use This Book

PART 0 · THE PLANS

0

How to Use This Book

A single path from first principles to the offer, with one home for every idea.

1. 0.1 What this book is —
2. 0.2 Tracks and tags —
3. 0.3 The 8-week plan —

0.1 What this book is

This book prepares a beginner for network-engineer and networking-software roles at Cisco, Qualcomm, Google, Microsoft, Amazon, Juniper, Arista, Nokia and Broadcom. It is designed to be the only resource one needs. Every concept is explained from first principles, every interview question is solved in full, and every lab prints the output one should expect, so that a lab can also be followed on paper.

Nothing is printed twice. Each concept, figure and solved question has exactly one home, and every other mention is a cross-reference that prints its page number. Thus, the book stays comprehensive without growing long, and a reader who meets a term out of order can always find its single, complete treatment.

> BIG IDEA
>
> Read the Must sections in order, solve the Problem Set at the end of each pillar, and revise from the two-page Revision Spread; the rest of the book is depth you add when time allows.

Each pillar follows the same order: (i) teaching sections, each opening with its Big Idea; (ii) a Problem Set, in which every reported interview question for the pillar is solved; and (iii) a Revision Spread, whose boxed top half is the version to re-read in the fifteen minutes before an interview.

0.2 Tracks, tags and the six elements

Roles fall into three tracks, which differ in what the interviews emphasise. The **Software** track covers networking software engineers at Cisco, Juniper, Arista, Nokia and Broadcom, and software engineers on networking teams at Google and Microsoft. The **Infrastructure** track covers network engineers, Amazon network development engineers and Cisco consulting engineers. The **Wireless** track covers WLAN and modem software roles, principally at Qualcomm.

Track — t
one is pre
students p

Pillar	Software	Infrastructure	Wireless
1 DSA & coding	●●●	●●	●●
2 C, Python, C++	●●●	●●	●●●
3 Computer Networks	●●●	●●●	●●●
4 Routing & Switching	●	●●●	●
5 Data Centre & Cloud	●●	●●●	●
6 Wireless & Cellular	—	●	●●●
7 OS & Sockets	●●●	●	●●●
8 Linux & Tools	●●	●●●	●●
9 Security	●	●●	●
10 Automation	●●	●●	—
11 Troubleshooting	●	●●●	●
12 Supporting subjects	●●●	●●●	●●●

Every section heading carries a tag such as `MUST · ■□□`. The word states priority: **Must** sections form the 8-week plan, **Should** sections are added by the 3-month plan, and **Optional** sections complete the 6-month plan. The squares state difficulty, from foundation (■□□) to the level of the hardest interviews (■■■). Note that the tags guide the order of reading; they do not change the content, which is written to the same standard throughout.

The text uses only six recurring elements. A **Big Idea** states what a section exists to teach. A **Worked Example** solves a problem step by step. A **Trap** names a common misconception and why it is wrong. An **Interview Lens** states how the topic is examined and which companies report asking it. A **Lab** gives commands and the output to expect. **Check Yourself** questions close a section, with answers in the footnotes. Definitions, analogies and mnemonics sit in the outer margin.

0.3 The 8-week plan

The plan assumes about 4 hours on weekdays and 6 hours on Saturdays. Each weekday has the same shape: 60 minutes of coding practice, 120 minutes of the day's Must sections, 40 minutes on a lab or a worked example redone without the text, and 20 minutes of recall. Sunday holds the weekly checkpoint and a buffer, because a plan with no slack fails at the first missed day.

Week	Main topic	Daily coding strand	Sunday
1	Layers, Ethernet, ARP, subnetting; C pointers and structs	Complexity, arrays, strings, hashing, bits	Checkpoint 1
2	TCP and UDP, congestion control, DNS, DHCP, "type google.com"; C traps	Linked lists, stacks, queues, sliding window	Checkpoint 2
3	VLANs, STP, the routing table, OSPF, BGP, ACLs and NAT	Trees, tries, heaps, BFS and DFS	Checkpoint 3, mock OA
4	Processes, threads, synchronisation, memory; sockets and epoll; Linux tools	Shortest paths, union-find, backtracking	Checkpoint 4
5	Spine-leaf, ECMP, VXLAN, cloud networks; TLS, IPsec, firewalls	Dynamic programming, greedy	Checkpoint 5
6	Track variant: troubleshooting and automation, or design coding, or Wi-Fi	LRU cache, rate limiter	Checkpoint 6, mock OA
7	Troubleshooting rounds; aptitude; OOP and DBMS	Timed mixed sets	Checkpoint 7, mock OA
8	Company playbook; behavioural stories; Revision Spreads	The top-30 list	Checkpoint 8, mock OA

> **TRAP**
>
> *Claim:* one should finish all the theory before starting coding practice. *Why it is wrong:* coding fluency is built by short, repeated practice spread over weeks, and it cannot be compressed

> into the final fortnight. *Correct statement:* code every day from the first day, in parallel with the networking chapters.